PROJECT REPORT

4-Bit 3-Stage Pipelined

RISC CPU Datapath

Design, Verification and FPGA Synthesis in Verilog HDL

Item	Details
Student name	Lam Duy Loc
Student ID	SE190907
Design language	Verilog HDL (IEEE 1364-2001/2005 style)
EDA tool	AMD/Xilinx Vivado 2026.1 (behavioral simulation, elaboration, synthesis)
Target device	xc7vx485tffg1157-1 (Virtex-7)
Report date	October 2026

Executive Summary

This report presents the complete design flow of a compact **4-bit RISC processor datapath** with a **3-stage pipeline** (Instruction Fetch, Instruction Decode, Execute/Memory/Write-back). The processor uses a 12-bit instruction word, four 4-bit general-purpose registers (R0–R3), a 16-word instruction ROM, a 16-word data RAM, a 7-operation ALU, Zero and Carry flags, and a hardwired control unit supporting **12 instructions** (arithmetic, logic, data movement, memory access and two jumps).

Control hazards created by taken branches are resolved in the Execute stage; the Branch Unit raises a `flush` signal that clears the two younger pipeline registers, so a taken `JMP` or `JZ` costs exactly **2 bubble cycles**. Data hazards are avoided by construction because register read, ALU operation, memory access and write-back all occur in the same Execute stage.

The design was verified by a hierarchy of self-checking testbenches culminating in a master regression of **88 checks (11 ALU + 11 Register File + 22 Control Unit + 44 CPU integration)**, all of which pass in the Vivado simulator (simulation ended at 866 ns). The same regression was independently re-run during the preparation of this report with Icarus Verilog and produced identical results. The design synthesizes cleanly for the Virtex-7 xc7vx485tffg1157-1 device using only **35 LUTs, 50 flip-flops and 28 I/O pins**.

Table 1. *Summary of principal results.*

Key result	Value
Architecture	4-bit data path, 12-bit instructions, 3-stage pipeline, hardwired control
Instruction set	12 implemented opcodes + 4 reserved opcodes decoded as NOP
Branch penalty	2 cycles per taken branch (flush of IF/ID and ID/EX registers)
Functional verification	88 / 88 checks passed (ALU 11, RF 11, CU 22, CPU 44)
Simulation length	866 ns (10 ns clock period)
Elaborated design	13 cells, 28 I/O ports, 116 nets
Synthesized design	30 cells, 28 I/O ports, 37 nets
Post-synthesis utilization	LUT 35 (0.01 %), FF 50 (0.01 %), I/O 28 (4.67 %)

Contents

1. Introduction
2. System Architecture
3. Instruction Set Architecture
4. Detailed Module Design
5. Pipeline Operation and Hazard Analysis
6. Functional Verification
7. Synthesis and Resource Utilization
8. Discussion, Limitations and Future Work
9. Conclusion

Appendix A. Project File Inventory
Appendix B. How to Reproduce the Results

1. Introduction

1.1 Background and Objectives

Understanding how a processor works is best achieved by building one. A small RISC-style CPU exposes every essential concept of computer architecture: instruction encoding, instruction decoding, register files, arithmetic logic, memory access, program-counter control, pipelining and hazard handling. This project implements such a CPU in Verilog HDL at a deliberately small scale (4-bit data, 4-bit addresses) so that every signal can be inspected in a waveform and every behaviour can be verified exhaustively.

The objectives of the project are:

1. Define a compact instruction set architecture (ISA) with a fixed 12-bit instruction format.
2. Implement a modular datapath consisting of a program counter, instruction memory, register file, ALU, flag register, data memory, multiplexers and pipeline registers.
3. Implement a hardwired control unit (instruction decoder and branch unit) that drives the datapath.
4. Organize the datapath as a three-stage pipeline and resolve control hazards by flushing.
5. Verify each module and the integrated CPU using self-checking testbenches and several assembly programs.
6. Synthesize the design in Vivado, inspect the elaborated and synthesized schematics, and report resource utilization.

1.2 Project Deliverables

The submitted archive `LamDuyLoc_SE190907_4bit_CPU_Datapath.zip` is organized into four folders, summarized in Table 2.

Table 2. *Contents of the submitted project archive.*

Folder	Contents	Size
`sources/`	11 synthesizable Verilog files: alu, control_unit (decoder + branch unit + top), cpu_top, data_memory, flag_register, instruction_memory, instruction_register, mux, pipeline_register, program_counter, register_file	≈ 750 lines
`sim/`	5 testbenches: tb_alu, tb_register_file, tb_control_unit, tb_cpu and the master regression tb_regression_88	≈ 1,000 lines
`programs/`	4 machine-code files (`.mem`, hexadecimal): program, prog_arith, prog_logic, prog_jump	46 instructions
`Screenshots/`	Vivado evidence: project sources, waveforms, Tcl console, elaborated schematic, synthesized schematic, utilization report, and three truth-table tables	12 images

1.3 Tools and Environment

- **Design entry:** Verilog HDL; all modules use synchronous active-high reset except where noted.
- **Simulator:** Vivado Simulator (behavioral, functional simulation, `sim_1`, top module `tb_regression_88`).
- **Synthesis:** Vivado 2026.1 targeting the Virtex-7 device `xc7vx485tffg1157-1`.

- **Cross-check:** Icarus Verilog 12.0, used while preparing this report to re-run all five testbenches independently.

1.4 Organization of This Report

Chapter 2 describes the overall architecture. Chapter 3 defines the instruction set. Chapter 4 documents each module. Chapter 5 analyzes pipeline behaviour and hazards with cycle-accurate traces. Chapter 6 presents the verification strategy, test programs and results. Chapter 7 reports synthesis results. Chapter 8 discusses design strengths, limitations and possible extensions, and Chapter 9 concludes.

2. System Architecture

2.1 Specification Summary

Table 3. *Architectural specification of the CPU.*

Parameter	Specification
Data width	4 bits
Instruction width	12 bits: `[11:8]` opcode, `[7:6]` Rd, `[5:4]` Rs, `[3:0]` immediate / address
General-purpose registers	4 × 4-bit (R0–R3), 2 asynchronous read ports, 1 synchronous write port
Instruction memory	16 words × 12 bits ROM, asynchronous read, initialized from a `.mem` file with `$readmemh`
Data memory	16 words × 4 bits RAM, synchronous write, asynchronous read
Program counter	4 bits (addresses 0–15), wraps from 15 to 0
ALU	7 operations: ADD, SUB, AND, OR, XOR, NOT, PASS
Status flags	Zero (Z) and Carry (C), each with an independent write enable
Pipeline	3 stages: IF, ID, EX (execute + memory access + write-back)
Control	Hardwired: combinational instruction decoder (ID) and branch unit (EX)
Hazard handling	Control hazards: flush of IF/ID and ID/EX on taken branch. Data hazards: none by design
Reset	Synchronous, active high; clears PC, register file, flags and pipeline registers
Simulation clock	10 ns period (100 MHz) in all testbenches

2.2 Block Diagram

Figure 1 shows the three pipeline stages and the principal data and control paths. Instructions flow from left to right. The only backward path is the branch feedback drawn in red: when a jump is taken in the EX stage, the Branch Unit selects the jump target through the PC multiplexer and simultaneously flushes the two pipeline registers behind it.

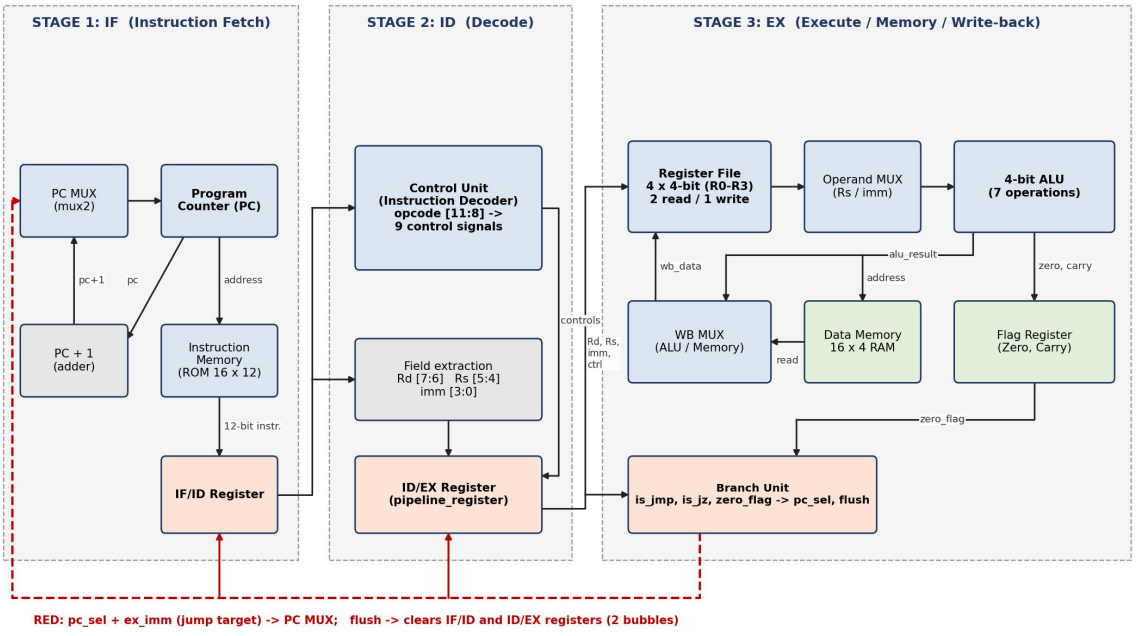


Figure 1. *Block diagram of the 3-stage pipelined datapath (drawn from the RTL in cpu_top.v).*

2.3 Pipeline Stages

Stage 1 – Instruction Fetch (IF)

The 4-bit program counter addresses the instruction ROM. A combinational adder computes `pc + 1`, and the PC multiplexer (`PC_MUX`) chooses between `pc + 1` and the branch target `ex_imm` according to `pc_sel`. The fetched instruction, together with its address and a `valid` bit, is captured by the IF/ID register (`instruction_register`) at the rising clock edge. The PC enable is tied high, so a new instruction is fetched on every cycle.

Stage 2 – Instruction Decode (ID)

The instruction held in the IF/ID register is split into opcode, Rd, Rs and immediate fields. The opcode drives the combinational instruction decoder, which produces the ALU control code and eight single-bit control signals. Decoded controls, register indices, the immediate, the instruction address, the valid bit and the raw instruction word are captured by the ID/EX register (`pipeline_register`).

Stage 3 – Execute, Memory Access and Write-back (EX)

In this stage the register file is read using Rd and Rs, the operand-B multiplexer selects Rs or the immediate, the ALU computes the result, and the flag register is updated if requested. Data-memory reads and writes also happen here, with the ALU result used as the address. The write-back multiplexer selects the ALU result or the memory read data, and the value is written back to Rd at the end of the cycle. The Branch Unit evaluates `is_jmp`, `is_jz` and the registered Zero flag to decide whether the pipeline must redirect.

> **Design decision: merging EX, MEM and WB**
> A classical five-stage pipeline would need forwarding or stalls because a result is written back several cycles after it is computed. By reading registers in the same stage in which results are written, this design guarantees that the next instruction always sees the updated register, flag and memory contents, which removes all data-hazard logic. The price is a longer combinational path in the EX stage (register read → ALU → data memory → write-back multiplexer → register input), which is acceptable for a 4-bit datapath.

2.4 Design Hierarchy

Figure 2 shows the Vivado Sources window after the project was loaded for simulation. The design hierarchy under `cpu_top` contains twelve instances, listed in Table 4.

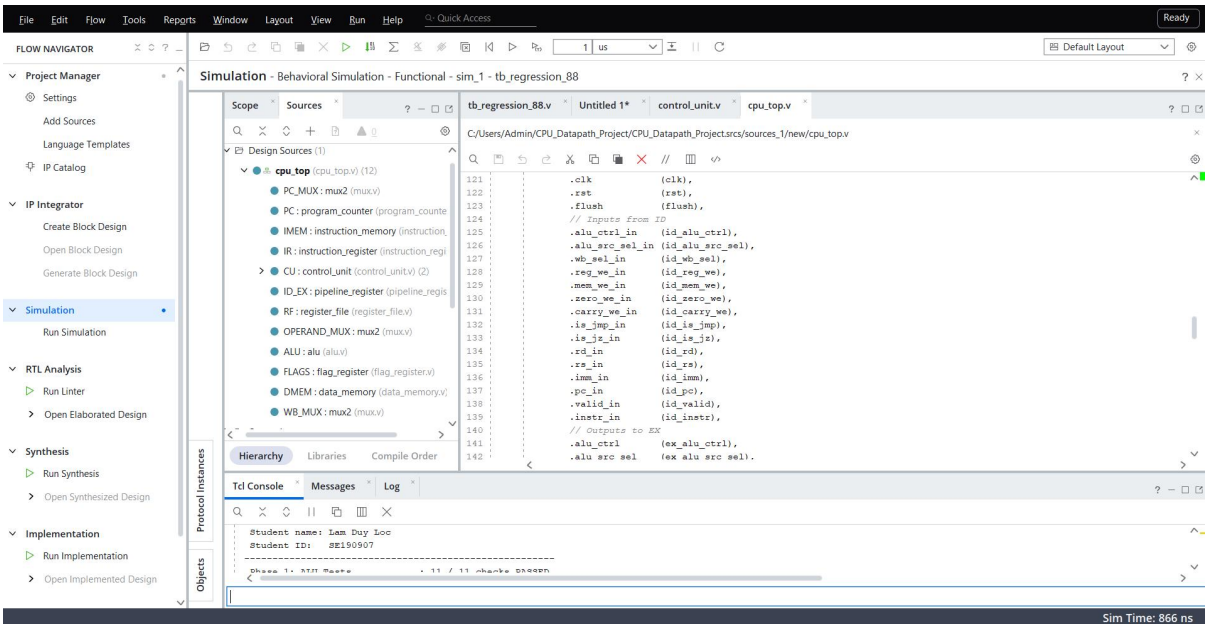


Figure 2. *Vivado project sources: cpu_top contains 12 instances; the Tcl console shows the regression result.*

Table 4. *Instances inside cpu_top.*

Instance	Module	Source file	Stage
`PC_MUX`	mux2	mux.v	IF
`PC`	program_counter	program_counter.v	IF
`IMEM`	instruction_memory	instruction_memory.v	IF
`IR`	instruction_register	instruction_register.v	IF/ID
`CU`	control_unit (contains `DECODER` and `BRANCH`)	control_unit.v	ID / EX
`ID_EX`	pipeline_register	pipeline_register.v	ID/EX
`RF`	register_file	register_file.v	EX
`OPERAND_MUX`	mux2	mux.v	EX
`ALU`	alu	alu.v	EX
`FLAGS`	flag_register	flag_register.v	EX
`DMEM`	data_memory	data_memory.v	EX
`WB_MUX`	mux2	mux.v	EX

2.5 Top-Level Interface

The top module `cpu_top` has a single parameter, `PROGRAM_FILE` (default `"program.mem"`), and exposes observable debug outputs so that internal behaviour can be seen on waveforms or on FPGA LEDs. The port list accounts for the 28 I/O ports reported by Vivado (2 inputs + 26 outputs).

Table 5. *Ports of cpu_top.*

Port	Dir.	Width	Description
`clk`	in	1	System clock

Port	Dir.	Width	Description
rst	in	1	Synchronous active-high reset
pc_out	out	4	Current fetch address (PC)
instr_out	out	12	Instruction currently in the EX stage (0xC00 when a bubble)
alu_out	out	4	ALU result
wb_data_out	out	4	Value presented to the register-file write port
zero_out	out	1	Registered Zero flag
carry_out	out	1	Registered Carry flag

3. Instruction Set Architecture

3.1 Instruction Format

All instructions are 12 bits wide and share one format. Fields that an instruction does not use are ignored by the hardware.

Table 6. *12-bit instruction format.*

Bits	11 – 8	7 – 6	5 – 4	3 – 0
Field	Opcode	Rd	Rs	Imm / Addr
Width	4 bits	2 bits	2 bits	4 bits
Meaning	Operation selector	Destination (and first source) register	Second source register	Immediate, data address or jump target

Arithmetic and logic instructions use a **two-operand** form: `Rd ← Rd op Rs`. Rd is therefore both a source and the destination, which is why the register file reads Rd on port A and Rs on port B while writing back to Rd.

3.2 Instruction Set

Table 7. *Complete instruction set.*

Opcode	Mnemonic	Operation	Flags	Description
0000	`ADD Rd,Rs`	Rd ← Rd + Rs	Z, C	4-bit addition; C = carry-out
0001	`SUB Rd,Rs`	Rd ← Rd − Rs	Z, C	Subtraction; C = 1 means no borrow (Rd ≥ Rs)
0010	`AND Rd,Rs`	Rd ← Rd & Rs	Z	Bitwise AND
0011	`OR Rd,Rs`	Rd ← Rd | Rs	Z	Bitwise OR
0100	`XOR Rd,Rs`	Rd ← Rd ^ Rs	Z	Bitwise XOR (XOR Rd,Rd clears a register)
0101	`NOT Rd`	Rd ← ~Rd	Z	Bitwise complement; Rs and imm ignored
0110	`LDI Rd,imm`	Rd ← imm	–	Load 4-bit immediate
0111	`MOV Rd,Rs`	Rd ← Rs	–	Register copy
1000	`LOAD Rd,addr`	Rd ← DMEM[addr]	–	Load from data memory (absolute address in imm)
1001	`STORE Rd,addr`	DMEM[addr] ← Rd	–	Store register to data memory
1010	`JMP target`	PC ← target	–	Unconditional jump (resolved in EX)
1011	`JZ target`	if Z = 1: PC ← target	–	Jump if the Zero flag is set
1100 – 1111	(reserved)	No operation	–	Decoded as NOP; `0xC00` is the canonical NOP

3.3 Flag Behaviour

Flags are written only by instructions whose `zero_we` or `carry_we` control bit is set. Because the two enables are independent, logic operations update Z but leave C unchanged, while data-movement and control-flow instructions leave both flags untouched. This lets a program test the result of an arithmetic operation after intervening `LDI`, `MOV`, `LOAD` or `STORE` instructions.

Table 8. *Flag update rules.*

Instruction group	Z updated	C updated	Notes
ADD, SUB	Yes	Yes	Z = (result = 0000); C = carry-out (ADD) or no-borrow (SUB)
AND, OR, XOR, NOT	Yes	No	C keeps its previous value

Instruction group	Z updated	C updated	Notes
LDI, MOV, LOAD, STORE	No	No	Register / memory instructions do not touch flags
JMP, JZ	No	No	JZ only reads Z

3.4 Encoding Example

Table 9 hand-assembles the six instructions of the demonstration program `program.mem`. Reading the hexadecimal word as binary and slicing it into the four fields gives the assembly instruction.

Table 9. *Machine code of program.mem decoded field by field.*

Addr	Hex	Binary	Opcode	Rd	Rs	Imm	Assembly
0	645	0110 0100 0101	0110	01 (R1)	00	0101	`LDI R1, 5`
1	683	0110 1000 0011	0110	10 (R2)	00	0011	`LDI R2, 3`
2	060	0000 0110 0000	0000	01 (R1)	10 (R2)	0000	`ADD R1, R2`
3	6CA	0110 1100 1010	0110	11 (R3)	00	1010	`LDI R3, 10`
4	470	0100 0111 0000	0100	01 (R1)	11 (R3)	0000	`XOR R1, R3`
5	A05	1010 0000 0101	1010	00	00	0101	`JMP 5`

4. Detailed Module Design

This chapter documents every module in the order in which an instruction travels through the pipeline. Identifier names (for example `mem`, `r0`–`r3`) are intentionally preserved so that testbenches can access internal state hierarchically, such as `CPU.RF.r1` or `CPU.DMEM.mem[0]`.

4.1 Program Counter and PC Multiplexer

`program_counter` is a 4-bit register with synchronous reset to 0000 and an enable input `pc_en`. In `cpu_top` the enable is tied to 1, so the PC changes on every rising clock edge. Its next value is produced by `PC_MUX` (a `mux2` instance): `pc_plus_1` when `pc_sel = 0`, or the jump target `ex_imm` when `pc_sel = 1`. Because the PC is only 4 bits wide it wraps naturally from 15 to 0, which gives a 16-instruction address space.

```
always @(posedge clk) begin
    if (rst)        pc_out <= 4'b0000;
    else if (pc_en) pc_out <= pc_next;   // PC+1 or jump target
end
```

4.2 Instruction Memory

`instruction_memory` is a 16 × 12-bit ROM with a combinational read (`assign instruction = mem[address]`). During initialization every word is first set to `12'hC00` (the NOP encoding using reserved opcode 1100) and then the program file given by the parameter `PROGRAM_FILE` is loaded with `$readmemh`. Pre-filling with NOPs means that a program shorter than 16 words runs safely: unused addresses execute as no-operations rather than as unknown (X) values. The array is called `mem` so that testbenches can reload a different program with `$readmemh(file, CPU.IMEM.mem)`.

4.3 IF/ID Pipeline Register

`instruction_register` stores the fetched instruction (12 bits), its address (4 bits) and a `valid_out` bit. When `rst` or `flush` is asserted it loads a bubble: instruction `C00`, address 0, `valid_out = 0`. Otherwise (with `ir_load` tied high) it captures the fetch-stage values. The `valid` bit travels down the pipeline and lets testbenches and waveform viewers distinguish a real instruction from a bubble.

4.4 Control Unit

`control_unit` is a wrapper around two purely combinational sub-modules: the **instruction decoder** in the ID stage and the **branch unit** in the EX stage. This split is the key to the pipeline's control design: decoding happens one stage earlier than branch resolution, and the decoded `is_jmp` / `is_jz` bits are carried to EX through the ID/EX register.

4.4.1 Instruction decoder

Every output is first assigned a safe default (all write enables 0, ALU = ADD), and the `case` statement overrides only what each opcode needs. As a result, reserved opcodes 1100–1111 and bubbles behave as true NOPs. The full decoder truth table is given below; it matches the control-unit testbench and the truth-table figure supplied with the project.

Table 10. *Instruction decoder truth table (ID stage).*

Opcode	Mnemonic	alu_ctrl	alu_src_sel	wb_sel	reg_we	mem_we	zero_we	carry_we	is_jmp	is_jz
0000	ADD	000	0	0	1	0	1	1	0	0
0001	SUB	001	0	0	1	0	1	1	0	0
0010	AND	010	0	0	1	0	1	0	0	0
0011	OR	011	0	0	1	0	1	0	0	0
0100	XOR	100	0	0	1	0	1	0	0	0
0101	NOT	101	0	0	1	0	1	0	0	0
0110	LDI	110	1	0	1	0	0	0	0	0
0111	MOV	110	0	0	1	0	0	0	0	0
1000	LOAD	110	1	1	1	0	0	0	0	0
1001	STORE	110	1	0	0	1	0	0	0	0
1010	JMP	000	0	0	0	0	0	0	1	0
1011	JZ	000	0	0	0	0	0	0	0	1
1100–1111	Reserved	000	0	0	0	0	0	0	0	0

Signal meanings: `alu_src_sel` 0 = operand B is Rs, 1 = operand B is the immediate; `wb_sel` 0 = write back the ALU result, 1 = write back the data-memory output. Note how `LOAD` and `STORE` reuse the ALU in PASS mode (code 110) with the immediate as operand B, so the **ALU result equals the memory address**.

4.4.2 Branch unit

The branch unit sees the instruction that is currently in EX. It asserts `pc_sel` for any `JMP`, and for a `JZ` when the registered Zero flag is 1. The `flush` output is simply a copy of `pc_sel`.

Table 11. *Branch unit truth table (EX stage).*

is_jmp	is_jz	zero_flag	pc_sel	flush	Action
0	0	0	0	0	PC + 1 (no branch)
0	0	1	0	0	PC + 1 (no branch)
0	1	0	0	0	JZ not taken, PC + 1
0	1	1	1	1	JZ taken, PC ← jump target
1	0	0	1	1	JMP taken, PC ← jump target
1	0	1	1	1	JMP taken, PC ← jump target
1	1	x	1	1	JMP has priority (never occurs: decoder never sets both)

```
always @(*) begin
    if (is_jmp)                 pc_sel = 1'b1;
    else if (is_jz && zero_flag) pc_sel = 1'b1;
    else                         pc_sel = 1'b0;
end
assign flush = pc_sel;   // clear the two younger pipeline registers
```

4.5 ID/EX Pipeline Register

`pipeline_register` is the widest register in the design. It carries all decoded controls plus the operands' indices and the immediate to the EX stage. On reset or flush every field is cleared (all write enables 0, `valid = 0`, `instr = C00`), so a flushed slot is a harmless bubble that cannot modify registers, memory or flags.

Table 12. *Signals stored in the ID/EX register.*

Signal	Width	Purpose
`alu_ctrl`	3	ALU operation select
`alu_src_sel`	1	Operand-B select (Rs / immediate)
`wb_sel`	1	Write-back select (ALU / memory)
`reg_we`, `mem_we`	1 + 1	Register-file and data-memory write enables
`zero_we`, `carry_we`	1 + 1	Flag write enables
`is_jmp`, `is_jz`	1 + 1	Branch type passed to the branch unit
`rd`, `rs`	2 + 2	Register indices
`imm`	4	Immediate / address / jump target (`ex_imm`)
`pc`	4	Address of the instruction (for tracing)
`valid`	1	0 marks a bubble
`instr`	12	Raw instruction word (drives `instr_out`)

4.6 Register File

`register_file` contains four 4-bit registers (`r0`–`r3`). Writing is synchronous: on the rising edge, if `we = 1`, the register selected by `wa` receives `wd`; a synchronous reset clears all four registers. Reading is combinational through two independent ports implemented as conditional chains, so the values are available during the same cycle. In `cpu_top` port A reads Rd, port B reads Rs, and the write address is Rd. Register R0 is an ordinary register (it is not hard-wired to zero).

4.7 Arithmetic Logic Unit

The ALU is a combinational block with two 4-bit operands, a 3-bit control code, a 4-bit result, a carry output and a zero output. Operand A always comes from Rd. Operand B is Rs, or the immediate when `alu_src_sel = 1`.

Table 13. *ALU function table.*

alu_ctrl	Operation	result[3:0]	carry	zero
000	ADD	A + B (low 4 bits)	Carry-out of A + B	result = 0000
001	SUB	A − B	1 if A ≥ B (no borrow)	result = 0000
010	AND	A & B	0	result = 0000
011	OR	A | B	0	result = 0000
100	XOR	A ^ B	0	result = 0000
101	NOT	~A	0	result = 0000
110	PASS	B	0	result = 0000
111	Default	0000	0	1

```
3'd0: {carry, result} = {1'b0, a} + {1'b0, b};   // ADD
3'd1: begin result = a - b;                       // SUB
          carry = (a >= b); end                  // 1 = no borrow
3'd5: begin result = ~a; carry = 1'b0; end        // NOT (A only)
3'd6: begin result = b;  carry = 1'b0; end        // PASS
assign zero = ~|result;                           // reduction NOR
```

Two design details are worth noting. First, the ADD carry is obtained by widening both operands to 5 bits, which yields a clean carry-out without extra logic. Second, SUB defines the carry as a **no-borrow** flag (set when A ≥ B), so after `SUB` the Zero flag means “equal” and the Carry flag means “unsigned greater-or-equal”. Code 111 is not generated by the decoder; the `default` branch forces a defined output (0000, carry 0) instead of leaving the outputs unassigned.

4.8 Flag Register

`flag_register` holds the Zero and Carry flags. Each flag has its own write enable, so ALU operations that do not define a carry (logic operations) can leave it unchanged. The flags are cleared by reset. The Branch Unit reads the **registered** Zero flag, which is already updated at the clock edge that ends the previous instruction's EX cycle; therefore a `JZ` placed immediately after an arithmetic or logic instruction sees the correct flag without any stall.

4.9 Data Memory

`data_memory` is a 16 × 4-bit RAM. It is cleared to zero in an `initial` block to avoid unknown values, written synchronously when `mem_we = 1` (address from the ALU result, data from Rd), and read combinationally. Because the read is asynchronous, a `LOAD` obtains its data in the same cycle and writes it to the register file through the write-back multiplexer; no load-use stall is needed.

4.10 Multiplexers

A single parameter-free 2-to-1, 4-bit module `mux2` (`assign y = sel ? d1 : d0`) is instantiated three times: `PC_MUX` (PC+1 / jump target), `OPERAND_MUX` (Rs / immediate) and `WB_MUX` (ALU result / memory data).

5. Pipeline Operation and Hazard Analysis

5.1 Normal Instruction Flow

In straight-line code one instruction enters the pipeline every clock cycle. An instruction fetched in cycle *k* is decoded in cycle *k + 1* and executed in cycle *k + 2*. Consequently the first instruction leaves the pipeline after three cycles, and **N sequential instructions complete in N + 2 cycles**, which is exactly what the testbench comments “9 instructions … pipeline: 9+2” and “12 instructions … pipeline: 12+2” state.

Table 14 is a cycle-by-cycle trace of the demonstration program, produced by re-simulating `program.mem`. The register values are those visible *during* the listed cycle, i.e. before the result of the EX-stage instruction is written at the end of the cycle. A dash denotes a bubble.

Table 14. *Pipeline trace of program.mem (instruction hex @ address).*

Cycle	IF (PC)	ID	EX	R0	R1	R2	R3	Z C	flush
1	0	–	–	0000	0000	0000	0000	00	0
2	1	645@0	–	0000	0000	0000	0000	00	0
3	2	683@1	645@0	0000	0000	0000	0000	00	0
4	3	060@2	683@1	0000	0101	0000	0000	00	0
5	4	6CA@3	060@2	0000	0101	0011	0000	00	0
6	5	470@4	6CA@3	0000	1000	0011	0000	00	0
7	6	A05@5	470@4	0000	1000	0011	1010	00	0
8	7	C00@6	A05@5	0000	0010	0011	1010	00	1
9	5	–	–	0000	0010	0011	1010	00	0
10	6	A05@5	–	0000	0010	0011	1010	00	0
11	7	C00@6	A05@5	0000	0010	0011	1010	00	1
12	5	–	–	0000	0010	0011	1010	00	0

The trace shows the pipeline filling during cycles 1–3. `ADD R1,R2` is in EX during cycle 5 and its result 1000 is visible from cycle 6; `XOR R1,R3` is in EX during cycle 7 and its result 0010 is visible from cycle 8. In cycle 8 the `JMP 5` is in EX, `flush` is asserted and the instruction that was in the ID stage (address 6, a NOP) is discarded. From cycle 9 onward the program repeats a three-cycle pattern: one useful `JMP` followed by two bubbles.

5.2 Control Hazards

A branch is not known to be taken until it reaches the EX stage. By then two younger instructions have already been fetched (one in IF, one in ID). When the Branch Unit asserts `pc_sel` and `flush`:

- the PC multiplexer selects `ex_imm`, so the next fetch is from the target address;
- the IF/ID register and the ID/EX register are both overwritten with bubbles at the next clock edge, discarding the two wrong-path instructions before they can execute;
- the branch instruction itself leaves EX normally and is **not** re-executed because the ID/EX register behind it is cleared.

The penalty is therefore a constant **2 cycles per taken branch**, and zero cycles for a `JZ` that is not taken. Figure 3 illustrates this for `prog_jump.mem`, which contains one unconditional jump, one taken `JZ` and one `JZ` that is not taken.

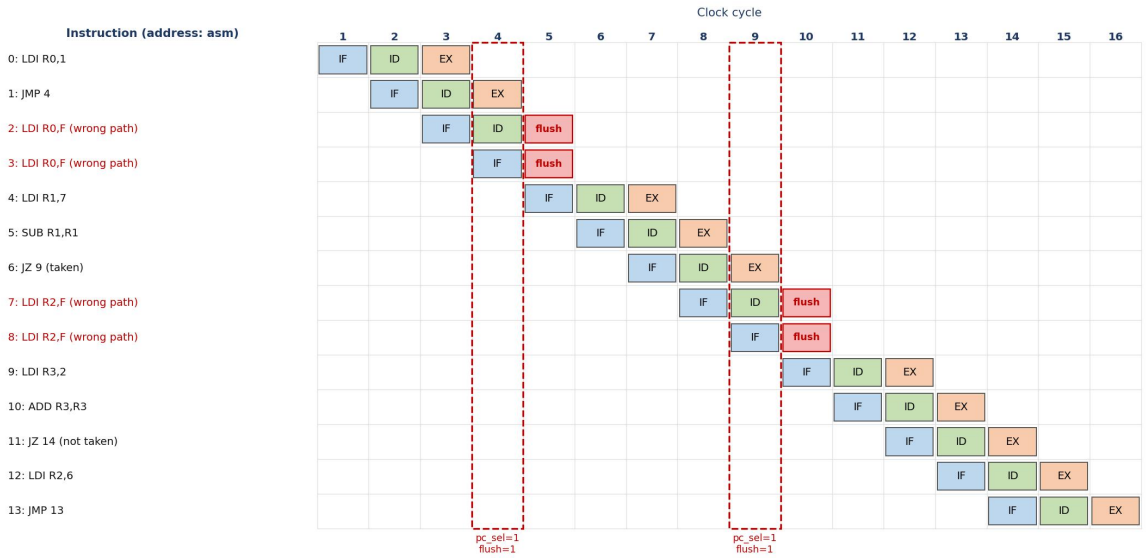


Figure 3. *Pipeline timing of prog_jump.mem: two taken branches each discard two wrong-path instructions (red).*

Reading the diagram: `JMP 4` reaches EX in cycle 4. In that cycle the instructions from addresses 2 and 3 occupy ID and IF; both are flushed, so the loads of `0xF` into R0 never execute and R0 keeps the value 1 written by address 0. The same happens at cycle 9 for `JZ 9` (the two `LDI R2,F` instructions at addresses 7 and 8 are discarded). `JZ 14` at cycle 14 finds Z = 0 and flows straight through, so `LDI R2,6` at address 12 executes and the program ends with R2 = 6.

5.3 Data and Structural Hazards

- **Read-after-write:** register read, ALU operation and write-back are all in EX. An instruction writing Rd at the end of cycle *t* is followed by an instruction whose EX cycle is *t + 1*, which reads the already-updated register. No forwarding network or interlock is needed. The same argument holds for the flag register and for a `STORE` followed by a `LOAD` of the same address.
- **Load-use:** data-memory read is combinational and happens in EX, so a loaded value is written back at the end of the same cycle; there is no load-use delay.
- **Flag-to-branch:** the flag is updated at the clock edge that ends the producing instruction's EX cycle, so a `JZ` immediately after `SUB` or `XOR` sees the new Z. This is exercised by `prog_jump.mem` (`SUB R1,R1` at address 5 immediately followed by `JZ 9`).
- **Structural:** instructions and data are held in separate memories (Harvard organization), the register file has two read ports and one write port, and the ALU is used once per instruction, so no resource is contended.

5.4 Throughput Analysis

If *f* is the fraction of instructions that are taken branches, the steady-state cycles per instruction is CPI = 1 + 2 f. The measured cycle counts for the four test programs, taken from the simulation trace and consistent with the testbench comments, are listed below. “Cycles” counts clock cycles from the first fetch until the last listed instruction has finished its EX cycle.

Table 15. *Cycle counts and CPI of the test programs.*

Program	Instructions executed	Cycles	Taken branches	CPI	Remark

Program	Instructions executed	Cycles	Taken branches	CPI	Remark
program.mem	6 (addr 0–5)	8	0 before the final JMP	1.33	Then loops: 1 useful + 2 bubble cycles per iteration
prog_arith.mem	10 (addr 0–9)	12	0 before the final JMP	1.20	Final `JMP 9` is a self-loop
prog_logic.mem	13 (addr 0–12)	15	0 before the final JMP	1.15	`JMP 14` lands in NOP area, PC wraps 15 → 0
prog_jump.mem	10 (addr 0,1,4,5,6,9–13)	16	2 (JMP 4, JZ 9)	1.60	4 bubble cycles + 2 fill cycles

Programs without a taken branch before the end reach CPI = (N + 2)/N, i.e. the 2 cycles are the one-time pipeline fill. `prog_jump.mem` reaches 1.60 because it executes two taken branches, each costing two bubbles, in just ten instructions.

6. Functional Verification

6.1 Verification Strategy

Verification follows a bottom-up approach. Each block with non-trivial behaviour (ALU, register file, control unit) has its own self-checking testbench. The integrated CPU is then tested with four assembly programs whose expected results were calculated by hand. Finally, a master regression (`tb_regression_88`) instantiates all four design-under-test blocks in one simulation and reports a per-phase and overall verdict. Every testbench prints the student name and ID in its banner, prints `PASS`/`FAIL` for each check, counts errors, and ends with `$finish`.

Table 16. *Testbench inventory.*

Testbench	Design under test	Checks	Purpose
`tb_alu.v`	alu	11	All seven ALU operations, carry and zero corner cases
`tb_register_file.v`	register_file	11	Reset, write, dual-port read, write-enable blocking
`tb_control_unit.v`	control_unit	22	16 opcodes (decoder) + 6 branch-unit cases
`tb_cpu.v`	cpu_top	44	Five integration tests with four programs, includes pipeline trace
`tb_regression_88.v`	alu, register_file, control_unit, cpu_top	88	Master regression: 11 + 11 + 22 + 44

6.2 Module-Level Tests

6.2.1 ALU (11 checks)

Each vector applies A, B and `alu_ctrl`, waits 10 ns and compares result, carry and zero.

Table 17. *ALU test vectors (all pass).*

Test	A	B	alu_ctrl	Result	C	Z
ADD 5 + 3	0101	0011	000	1000	0	0
ADD 15 + 1 (overflow)	1111	0001	000	0000	1	1
SUB 5 − 3	0101	0011	001	0010	1	0
SUB 3 − 5 (borrow)	0011	0101	001	1110	0	0
SUB 7 − 7	0111	0111	001	0000	1	1
AND	1100	1010	010	1000	0	0
OR	1100	1010	011	1110	0	0
XOR	1100	1010	100	0110	0	0
XOR to zero	0110	0110	100	0000	0	1
NOT 0101	0101	0000	101	1010	0	0
PASS B	0000	1001	110	1001	0	0

The vectors cover the interesting boundaries: carry-out on overflow, borrow on negative difference, and a zero result produced by three different operations.

6.2.2 Register file (11 checks)

- Reset: R0 (port A) and R3 (port B) read 0000 after reset (2 checks).
- Writes of 0001, 0101, 1010, 1111 to R0–R3, then dual-port reads: R0/R1, R2/R3, and R1 on both ports (6 checks).

- `we = 0` must not modify R2 (1 check).
- A second reset clears R3 and R2 (2 checks).

6.2.3 Control unit (22 checks)

Sixteen checks compare the packed 11-bit control vector `{alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we, zero_we, carry_we, is_jmp, is_jz}` for every opcode 0000–1111, including the four reserved opcodes. Six further checks drive the branch unit with all meaningful combinations of `is_jmp`, `is_jz` and `zero_flag` and verify that `pc_sel` and `flush` are equal and correct.

6.3 Test Programs

Four programs exercise the CPU. Hexadecimal words are listed as stored in the `.mem` files; the assembly was derived by decoding the fields as described in Chapter 3.

Program 1 – program.mem (demonstration)

Table 18. *program.mem.*

Addr	Hex	Assembly	Expected effect
0	645	LDI R1, 5	R1 = 0101
1	683	LDI R2, 3	R2 = 0011
2	060	ADD R1, R2	R1 = 1000, C = 0, Z = 0
3	6CA	LDI R3, 10	R3 = 1010
4	470	XOR R1, R3	R1 = 0010 (1000 ^ 1010)
5	A05	JMP 5	Self-loop; PC stays at 5

Program 2 – prog_arith.mem (arithmetic and memory)

Table 19. *prog_arith.mem.*

Addr	Hex	Assembly	Expected effect
0	645	LDI R1, 5	R1 = 0101
1	683	LDI R2, 3	R2 = 0011
2	060	ADD R1, R2	R1 = 1000, C = 0, Z = 0
3	940	STORE R1, 0	DMEM[0] = 1000
4	645	LDI R1, 5	R1 = 0101
5	160	SUB R1, R2	R1 = 0010, C = 1 (no borrow), Z = 0
6	941	STORE R1, 1	DMEM[1] = 0010
7	800	LOAD R0, 0	R0 = 1000 (read back)
8	1A0	SUB R2, R2	R2 = 0000, Z = 1
9	A09	JMP 9	Self-loop

Program 3 – prog_logic.mem (logic, MOV, NOT, LDI)

Table 20. *prog_logic.mem.*

Addr	Hex	Assembly	Expected effect
0	64C	LDI R1, 12	R1 = 1100
1	68A	LDI R2, 10	R2 = 1010
2	7D0	MOV R3, R1	R3 = 1100
3	2E0	AND R3, R2	R3 = 1000
4	7D0	MOV R3, R1	R3 = 1100

Addr	Hex	Assembly	Expected effect
5	3E0	OR R3, R2	R3 = 1110
6	460	XOR R1, R2	R1 = 0110
7	790	MOV R2, R1	R2 = 0110
8	605	LDI R0, 5	R0 = 0101
9	500	NOT R0	R0 = 1010
10	6CA	LDI R3, 10	R3 = 1010
11	4F0	XOR R3, R3	R3 = 0000, Z = 1
12	A0E	JMP 14	Jumps into the NOP area (see Section 8.2)

Program 4 – prog_jump.mem (jumps and flags)

Table 21. *prog_jump.mem.*

Addr	Hex	Assembly	Expected effect
0	601	LDI R0, 1	R0 = 0001
1	A04	JMP 4	Taken; addresses 2–3 skipped
2	60F	LDI R0, 15	Never executed (skipped)
3	60F	LDI R0, 15	Never executed (skipped)
4	647	LDI R1, 7	R1 = 0111
5	150	SUB R1, R1	R1 = 0000, Z = 1, C = 1
6	B09	JZ 9	Taken (Z = 1); addresses 7–8 skipped
7	68F	LDI R2, 15	Never executed (skipped)
8	68F	LDI R2, 15	Never executed (skipped)
9	6C2	LDI R3, 2	R3 = 0010
10	0F0	ADD R3, R3	R3 = 0100, Z = 0, C = 0
11	B0E	JZ 14	Not taken (Z = 0)
12	686	LDI R2, 6	R2 = 0110
13	A0D	JMP 13	Self-loop; program stops here
14	68F	LDI R2, 15	Never executed (JZ 14 not taken)

6.4 CPU Integration Tests (44 checks)

The testbench waits for a given instruction address to be in the EX stage (`ex_valid && ex_pc == addr`), advances to the next clock edge, and then inspects registers, memory, flags or the PC. This makes the checks independent of the exact cycle number and robust to pipeline bubbles.

Table 22. *Integration tests and what each verifies.*

Test	Program	Checks	Verified behaviour
1	program.mem	6	ADD result; LDI; XOR; R2 preserved; JMP sets PC to 5; PC keeps looping at 5
2	prog_arith.mem	10	ADD/SUB results and flags (C, Z); STORE to DMEM[0] and [1]; LOAD back to R0; SUB R2,R2 sets Z
3	prog_logic.mem	8	AND, OR, XOR, MOV, NOT, LDI; XOR R3,R3 sets Z
4	prog_jump.mem	11	PC after reset (0) and increment (1); JMP 4 target and ID/EX flush; SUB sets Z; JZ taken (PC = 9); ADD clears Z; JZ not taken (EX address 12); skipped instructions had no effect (R0 = 1, R2 = 6); stop at PC = 13
5	(reset)	9	Reset clears PC, R0, R2, Z, C, IF/ID valid, ID/EX valid (7 checks); no X/Z values on outputs or registers after running 6 cycles (1 check), plus the PC check

Between programs the testbench holds reset, clears both memories and loads the next `.mem` file hierarchically (`$readmemh(file, CPU.IMEM.mem)`), which allows several programs to be tested in a single simulation run.

6.5 Simulation Results

Figure 4 shows the Tcl console at the end of the regression. All four phases report a full pass and the overall verdict is **PASSED (88 of 88 checks OK)**; the simulation finishes at 866 ns.

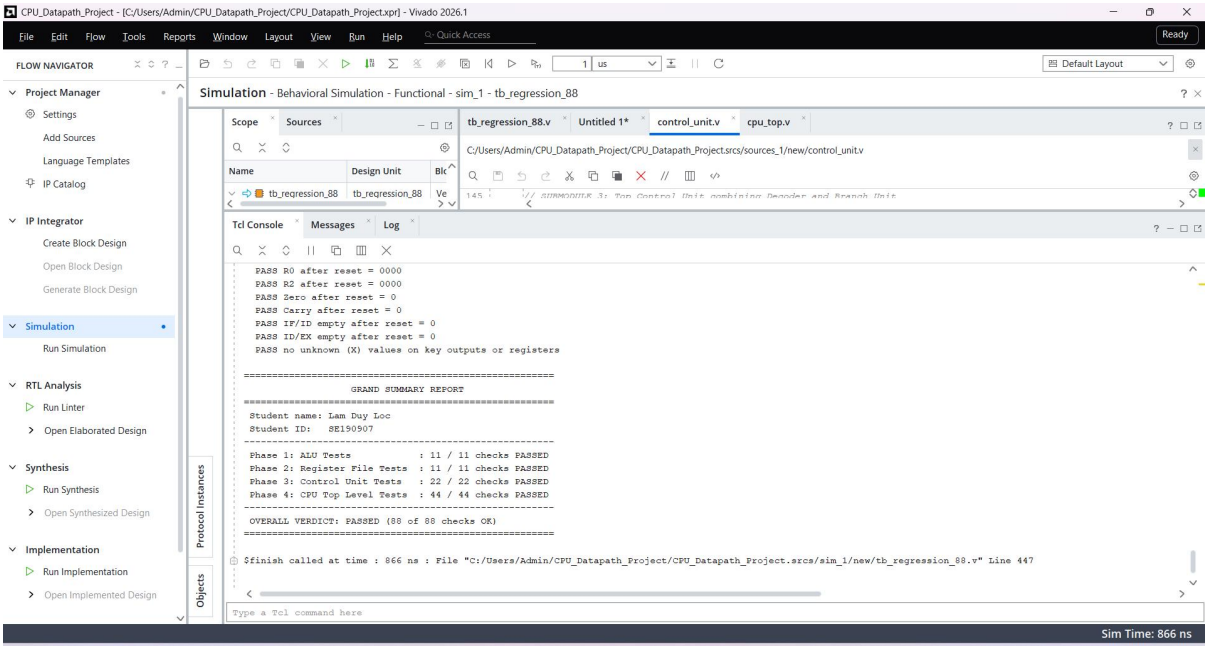


Figure 4. *Grand summary printed by tb_regression_88 in the Vivado Tcl console.*

Table 23. *Regression results by phase.*

Phase	Block	Passed	Total	Verdict
1	ALU	11	11	PASSED
2	Register File	11	11	PASSED
3	Control Unit	22	22	PASSED
4	CPU Top Level	44	44	PASSED
Overall		88	88	PASSED

Independent cross-check

While preparing this report the five testbenches were also compiled and run with Icarus Verilog 12.0 using the unmodified source files. `tb_alu` (11/11), `tb_register_file` (11/11), `tb_control_unit` (22/22), `tb_cpu` (44/44) and `tb_regression_88` (88/88) all passed, and the regression ended at 866 ns, identical to the Vivado result. The cycle-by-cycle traces in Chapter 5 were generated in the same way.

6.6 Waveform Analysis

Figure 5 shows the complete 866 ns run. During the first ≈215 ns the CPU is held in reset (`rst = 1`) while phases 1–3 of the regression exercise the ALU, register file and control unit; therefore `pc` stays at 0 and `id_instr` shows the bubble value `c00`. After reset is released the CPU runs `program.mem`, and short `rst` pulses later in the run mark the resets issued before each new

program and before the dedicated reset test. The `pc_sel` and `flush` traces pulse only when a jump is taken.

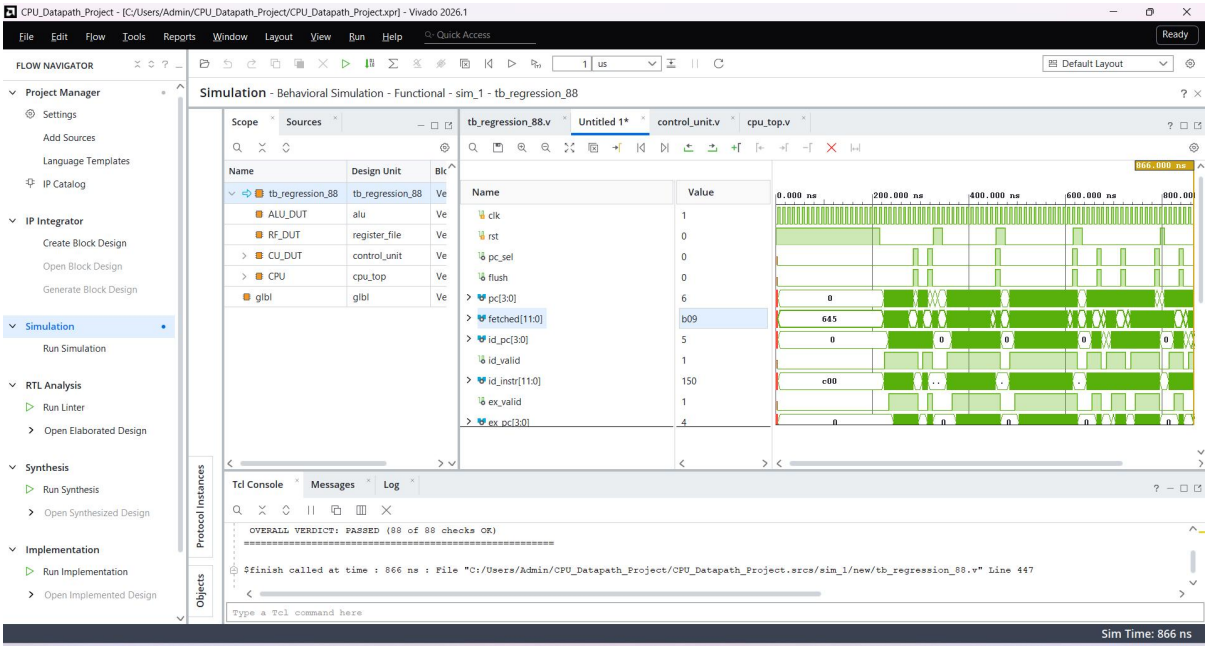


Figure 5. *Full-length waveform of the regression (0 – 866 ns).*

Figure 6 zooms into 200–360 ns. After reset is released the PC counts 0, 1, 2, 3, 4, 5 and the fetched words are 645, 683, 060, 6CA, 470 and A05, exactly the machine code of `program.mem`. The PC then goes 6, 7 and returns to 5: the fetched words at addresses 6 and 7 are the NOP value C00 and are the two wrong-path instructions discarded by the flush. In the same interval `pc_sel` and `flush` are high for one clock period each time `JMP 5` is in the EX stage, `id_valid` and `ex_valid` drop to 0 (bubbles), and `ex_pc` shows the address of the instruction being executed. Around 330 ns the testbench reloads the next program and asserts `rst`; the brief appearance of `160` on `fetched` is address 5 of `prog_arith.mem`, after which the PC returns to 0 and the identical first three words 645, 683, 060 are fetched.

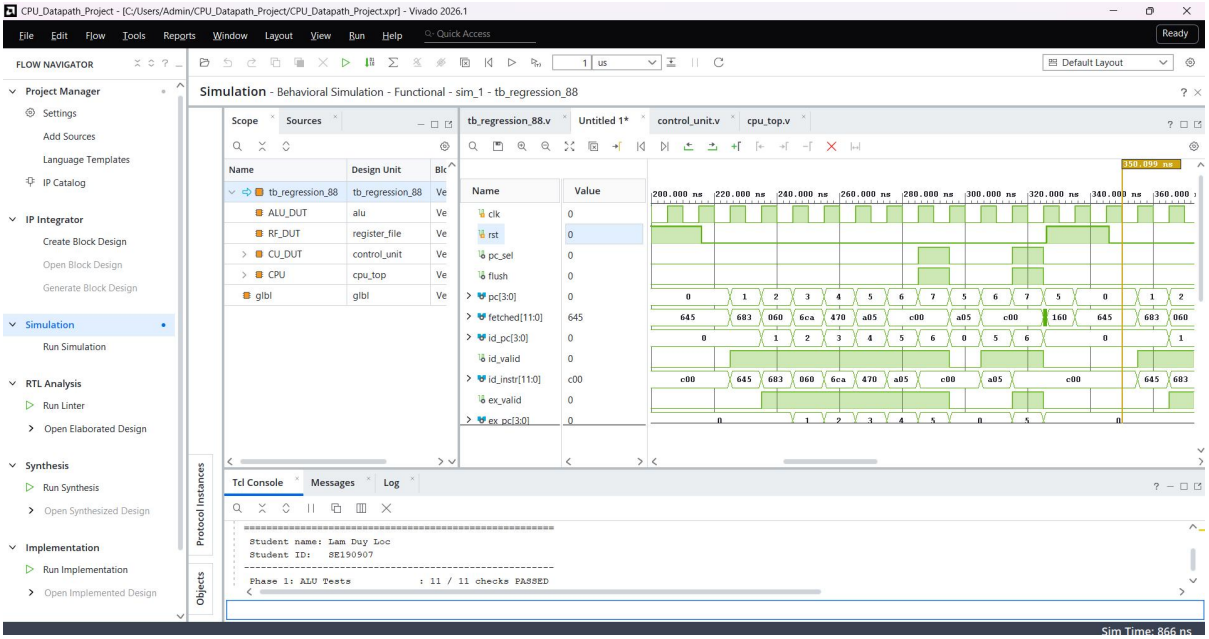


Figure 6. *Zoomed waveform (200 – 360 ns): JMP-loop with flush pulses and program switch.*

7. Synthesis and Resource Utilization

7.1 Elaborated (RTL) Design

Elaboration in Vivado converts the Verilog into a technology-independent netlist. Figure 7 shows the result for the device `xc7vx485tffg1157-1`: **13 cells, 28 I/O ports and 116 nets**. The 13 cells are the 12 hierarchical instances listed in Table 4 plus the RTL adder that computes `pc_plus_1`. The 28 I/O ports are `clk`, `rst` and the 26 observable output bits. The schematic confirms the intended structure: the instruction memory feeds the IF/ID register, the control unit decodes into the ID/EX register, and the register file, operand multiplexer, ALU, flags, data memory and write-back multiplexer sit in the EX stage with the branch feedback returning to `PC_MUX`.

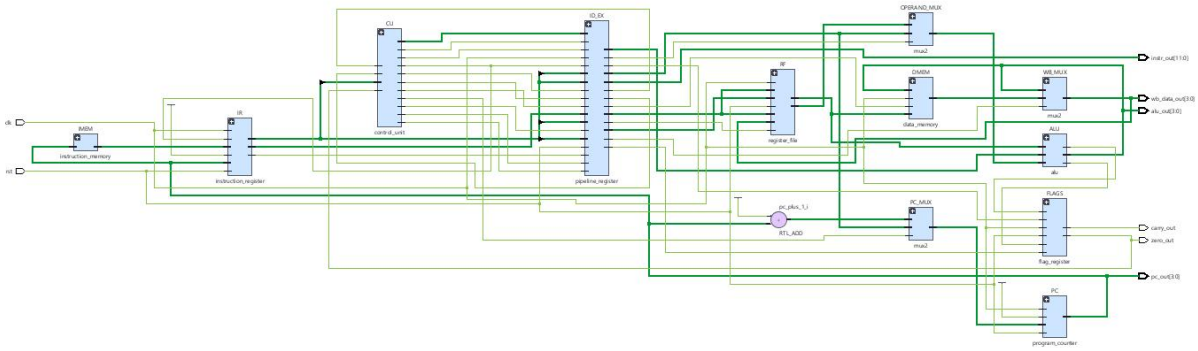


Figure 7. *Elaborated schematic of cpu_top (13 cells, 28 I/O ports, 116 nets).*

7.2 Synthesized Design

After synthesis the netlist is mapped onto device primitives. The synthesized top-level schematic (Figure 8) reports **30 cells, 28 I/O ports and 37 nets**; the visible part shows the input and output buffers on every port and the `PC` instance driving `pc_out`. Vivado displays “Synthesis Complete” and the netlist browser lists 30 leaf cells.

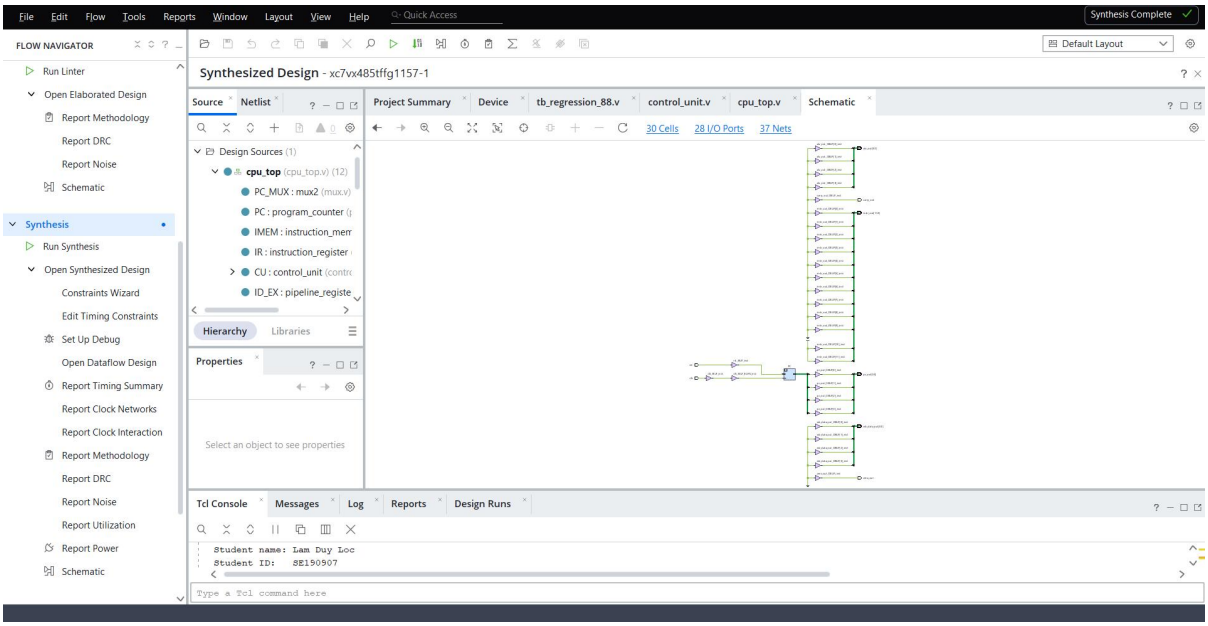


Figure 8. *Synthesized design schematic (30 cells, 28 I/O ports, 37 nets).*

7.3 Resource Utilization

The utilization summary (Figure 9) reports the resources consumed on the xc7vx485t device, which offers 303,600 LUTs, 607,200 flip-flops and 600 user I/O pins.

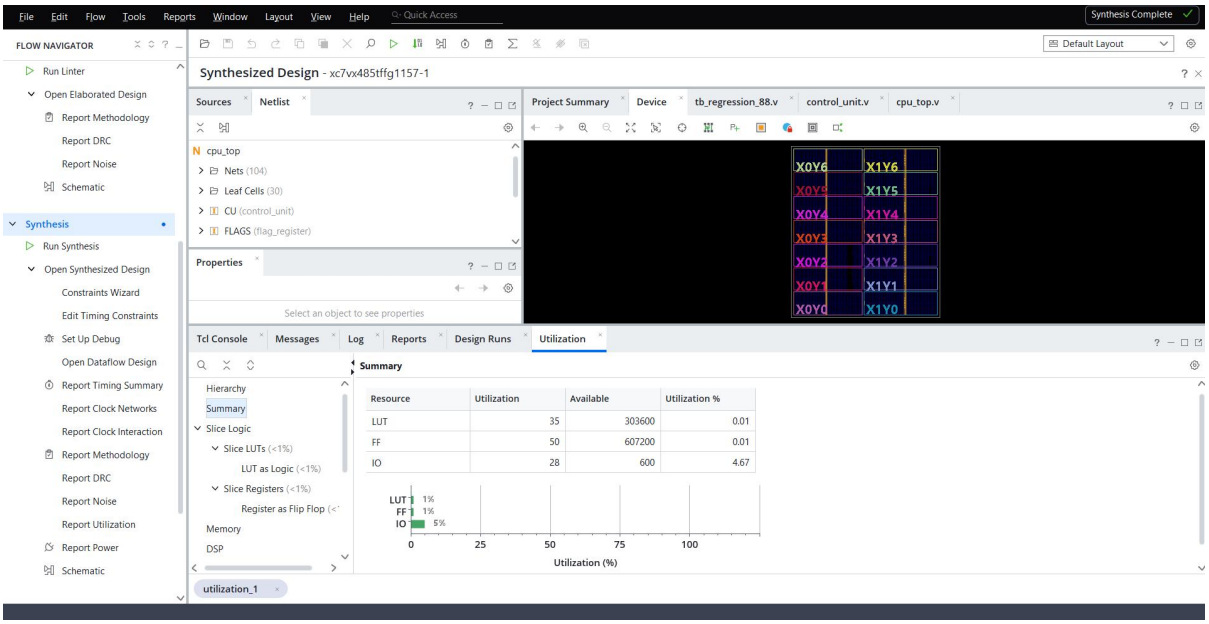


Figure 9. *Post-synthesis utilization summary.*

Table 24. *Post-synthesis resource utilization.*

Resource	Used	Available	Utilization
LUT	35	303,600	0.01 %
Flip-flop (FF)	50	607,200	0.01 %
I/O	28	600	4.67 %

7.4 Interpretation

- **Tiny footprint.** A 4-bit CPU uses a negligible fraction of a large device; the dominant resource is I/O because the debug outputs are brought out to pins.
- **Flip-flop count.** Counting the architectural registers gives 75 bits (PC 4, IF/ID 17, ID/EX 36, register file 16, flags 2). The synthesizer reported 50, which most likely reflects normal optimization: removal of bits that never influence an output (for example the pipeline copy of the instruction address, which is only used for tracing) and merging of registers that hold identical values. This is an interpretation of the report, not a figure printed by the tool.
- **Memories.** Only LUT, FF and I/O appear in the utilization summary; no block RAM or DSP resources are reported. With the default `program.mem` fixed in the ROM and no `STORE` in it, the tool can also simplify logic that depends on constant memory contents.
- **Scope of the data.** The reports supplied are post-synthesis only. No timing constraints, timing summary, power report or implementation results were included, so a maximum clock frequency cannot be stated; 100 MHz is only the simulation clock.

8. Discussion, Limitations and Future Work

8.1 Strengths of the Design

- **Clean modularity.** Each function is a separate, small module; the hierarchy maps one-to-one to the block diagram and is easy to inspect in Vivado.
- **Safe by default.** The decoder's default assignments, the NOP-filled instruction ROM, the cleared data RAM, and bubble-clearing pipeline registers prevent unknown values and unintended side effects. The regression explicitly checks that no X values appear.
- **Hazard-free data path.** Executing, accessing memory and writing back in a single stage eliminates forwarding and stall logic.
- **Strong verification.** Module tests, four programs, a reset test and a master regression give an 88-check automated suite; checks are keyed on instruction addresses rather than cycle numbers, so they remain valid if timing changes.

8.2 Observations

- **Program wrap-around.** `prog_logic.mem` ends with `JMP 14`. Addresses 14 and 15 contain the default NOP, so after the jump the PC advances 14 → 15 → 0 (4-bit wrap) and the program silently restarts from address 0. The testbench only checks up to address 11, so this does not affect the results, but a self-loop such as `JMP 12` would be a cleaner end-of-program marker.
- **Carry flag usage.** The Carry flag is computed and visible on `carry_out`, but no instruction branches on it; only `JZ` reads the Zero flag.
- **ALU code 111** is unreachable from the decoder and exists only as a defined default.
- **Reserved opcodes** 1100–1111 are decoded as NOP and 0xC00 is used as the bubble encoding, so software that executes a reserved opcode is silently ignored rather than trapped.

8.3 Limitations

- Only 16 instruction words and 16 data words are addressable (4-bit PC and addresses), and branch targets are absolute 4-bit values.
- Each taken branch costs 2 cycles; there is no branch prediction or delay slot.
- Only unconditional jump and jump-if-zero are provided; there are no shift, increment, compare-immediate or call/return instructions, and immediates can only be loaded with `LDI`.
- The long EX-stage combinational path will limit the achievable clock frequency; this was not measured because no timing report is included.
- The instruction ROM is initialized with `$readmemh`, which is appropriate for simulation and FPGA initialization but does not model an externally programmable memory.

8.4 Possible Improvements

- Add conditional branches on Carry (`JC`, `JNC`) and `JNZ`, and shift/rotate instructions, to make use of the existing flags.
- Move branch resolution to the ID stage or add a small branch predictor to reduce the 2-cycle penalty.

- Split EX into EX and MEM/WB to shorten the critical path, adding forwarding paths, and measure the effect with a timing report.
- Add an XDC constraints file and run implementation to obtain timing, power and placed-and-routed utilization; map `pc_out`/`alu_out` to board LEDs for a hardware demonstration.
- Extend the testbenches with random instruction streams compared against a software reference model, and add tests for `STORE`/`LOAD` at the highest addresses and for borrow flags at CPU level.

9. Conclusion

This project delivered a complete, working 4-bit RISC CPU datapath, from instruction-set definition through RTL design, verification and synthesis. The processor implements twelve instructions on a three-stage pipeline, handles taken branches by flushing two pipeline registers (a fixed two-cycle penalty), and avoids data hazards by performing register read, execution, memory access and write-back in one stage.

Verification is thorough for a design of this size: 88 automated checks pass in the Vivado simulator, the results were reproduced independently with a second simulator, and waveforms and cycle-accurate traces confirm the pipeline and flush behaviour predicted by the architecture. Synthesis for the Virtex-7 xc7vx485tffg1157-1 uses only 35 LUTs, 50 flip-flops and 28 I/O pins.

The work therefore meets all stated objectives. The main opportunities for further study are timing analysis and implementation on a board, reducing the branch penalty, and enlarging the instruction set and address space.

Appendix A. Project File Inventory

Table 25. *Source, simulation and program files.*

File	Module(s)	Role
`sources/alu.v`	alu	4-bit ALU with carry and zero outputs
`sources/control_unit.v`	instruction_decoder, branch_unit, control_unit	Hardwired control
`sources/cpu_top.v`	cpu_top	Top-level integration of the 3-stage pipeline
`sources/data_memory.v`	data_memory	16 × 4 RAM
`sources/flag_register.v`	flag_register	Zero and Carry flags
`sources/instruction_memory.v`	instruction_memory	16 × 12 ROM loaded from `.mem`
`sources/instruction_register.v`	instruction_register	IF/ID pipeline register
`sources/mux.v`	mux2	2-to-1 4-bit multiplexer
`sources/pipeline_register.v`	pipeline_register	ID/EX pipeline register
`sources/program_counter.v`	program_counter	4-bit PC
`sources/register_file.v`	register_file	4 × 4 register file
`sim/tb_alu.v`, `tb_register_file.v`, `tb_control_unit.v`	—	Module-level testbenches
`sim/tb_cpu.v`	tb_cpu	CPU integration testbench with pipeline trace
`sim/tb_regression_88.v`	tb_regression_88	Master 88-check regression
`programs/*.mem`	—	program, prog_arith, prog_logic, prog_jump
`Screenshots/`	—	F1 sources, F2 waveforms, F3 Tcl console, F4 elaborated, F5 synthesized, F6 utilization, truth tables 1–3

Appendix B. How to Reproduce the Results

B.1 Vivado

7. Create an RTL project for device `xc7vx485tffg1157-1`; add all files in `sources/` as design sources and the files in `sim/` as simulation sources.
8. Add the four `.mem` files to the project (simulation sources) so that `$readmemh` can find them, and set `tb_regression_88` as the simulation top.
9. Run Behavioral Simulation and run to completion (`run all`); the Tcl console prints the Grand Summary Report.
10. Run RTL Analysis → Open Elaborated Design for the schematic, then Run Synthesis → Open Synthesized Design → Report Utilization.

B.2 Icarus Verilog (command line)

```
cd programs   # so that the .mem files are found
iverilog -g2005 -o reg ../sources/*.v ../sim/tb_regression_88.v
vvp reg                       # expect: OVERALL VERDICT: PASSED (88 of 88 checks OK)

for t in tb_alu tb_control_unit tb_register_file tb_cpu; do
    iverilog -g2005 -o $t ../sources/*.v ../sim/$t.v && vvp $t
done
```